Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

08. Kubernetes Local (Minikube)

Design de API, DTOs e validação

De GETs funcionais a um contrato HTTP previsível.



Percurso da aula

01 **Contrato HTTP**
Operações, status e efeito de repetir um pedido

02 **Dados de entrada**
DTOs, leitura do JSON e regras de validação

03 **Resposta da API**
Mapping, tipos de retorno e ResponseEntity

04 **Prática e checkpoint**
Pedido válido, erro 400 e revisão do contrato

Objetivo: criar um filme com um contrato separado do modelo interno.

C2 General

Rumos

Consultar é insuficiente: o recurso tem um ciclo de vida



URI e métodos HTTP

GET	`/api/movies`	consultar coleção
POST	`/api/movies`	criar recurso
GET	`/api/movies/{id}`	consultar recurso
PUT	`/api/movies/{id}`	substituir recurso
PATCH	`/api/movies/{id}`	alterar parte do recurso
DELETE	`/api/movies/{id}`	remover recurso

Status codes contam o que aconteceu

200 OK

consulta/update com representação

201 Created

novo recurso criado

204 No Content

operação concluída sem body

400 Bad Request

request inválido

404 Not Found

recurso não existe

500 Internal Server Error

falha inesperada no servidor

401, 403 e outros erros frequentes em APIs

O código descreve o resultado HTTP. O corpo pode explicar o problema ao cliente.

Código	Nome HTTP	Quando usar ou interpretar
401	**Unauthorized**	Faltam credenciais válidas. Ex.: token ausente ou expirado.
403	**Forbidden**	O servidor recusa a operação. Ex.: utilizador sem permissão.
405	**Method Not Allowed**	O recurso existe, mas não aceita este método HTTP.
409	**Conflict**	O pedido entra em conflito com o estado atual do recurso.
415	**Unsupported Media Type**	O formato do corpo não é suportado. Ex.: XML numa API JSON.
429	**Too Many Requests**	O cliente excedeu o limite de pedidos permitido.
503	**Service Unavailable**	O serviço está temporariamente indisponível ou sobrecarregado.

401: autenticação necessária ou inválida. 403: acesso recusado, mesmo podendo estar autenticado.

C2 General

Idempotência: o efeito de um pedido repetido

Um pedido ou vários pedidos idênticos produzem o mesmo efeito pretendido no servidor.

```
DELETE /api/movies/1
```

Pedido	Resposta possível	Estado do recurso
Primeira execução	204 No Content	Filme 1 ausente
Repetição do mesmo pedido	404 Not Found	Filme 1 continua ausente

As respostas podem diferir. O efeito pretendido mantém-se.

Quais métodos HTTP são idempotentes?

Método	Idempotente?	Porquê
GET, HEAD, OPTIONS, TRACE	Sim	São métodos seguros: não pedem uma alteração do recurso.
PUT	Sim	O mesmo URI e a mesma representação fixam o mesmo estado.
DELETE	Sim	Remover o mesmo recurso mantém-no ausente.
POST	Sem garantia	Repetir pode criar outro recurso ou executar outra ação.
PATCH	Sem garantia	Depende da alteração: fixar um valor difere de incrementá-lo.

Seguro: não pede uma alteração. Idempotente: repetir mantém o efeito.

Rumos
C2 General

POST e PATCH: a operação concreta faz a diferença

PATCH: fixar um valor

`Definir year = 2024`

Após uma execução: 2024

Após duas execuções: 2024

Esta alteração é idempotente.

PATCH: incrementar um valor

`Somar 1 a year`

Valor inicial: 2023

Após uma execução: 2024

Após duas execuções: 2025

POST: criar um novo filme

Repetir POST /api/movies pode criar dois filmes com IDs diferentes, mesmo que o corpo JSON seja igual.

Repetições seguras em POST exigem um contrato explícito de deduplicação.

C2 General

Que dados do filme deve a API enviar?

Contrato da API: o formato dos dados que o cliente envia e recebe.
O cliente pode ser uma aplicação, um site ou outro sistema.

- **Campos do objeto Movie**

```
id = 1
title = "Matrix"
year = 1999
notaInterna = "rever"
```

JSON ao devolver Movie diretamente

```
{
  "id": 1,
  "title": "Matrix",
  "year": 1999,
  "notaInterna": "rever"
}
```

A nota interna serve à equipa. Devolvê-la ao cliente expõe um detalhe interno.

Um formato para o pedido, outro para a resposta

DTO (Data Transfer Object): um objeto com os dados que escolhemos trocar pela API.

1. O cliente envia

`MovieRequest`

```
{
 "title": "Matrix",
 "year": 1999
}
```

2. A aplicação trabalha

`Movie`

```
id = 1
title = "Matrix"
year = 1999
notaInterna = "rever"
```

3. A API responde

`MovieResponse`

```
{
 "id": 1,
 "title": "Matrix",
 "year": 1999
}
```

A aplicação atribui o id. A resposta inclui o id e deixa a nota interna de fora.

O mapeamento copia os campos escolhidos entre Movie e os DTOs.

MovieRequest e MovieResponse: os formatos do contrato

```java
public record MovieRequest(
    String title,
    Integer year
) {}
```

```java
public record MovieResponse(
    Long id,
    String title,
    int year
) {}
```

cliente envia title e year

API acrescenta o id na resposta

O DTO define os campos. A validação acrescenta regras aos valores recebidos.

O pedido HTTP transporta os dados do filme

O cliente preenche um título e um ano. Para criar o filme, envia estes dados à API.

Exemplo de pedido do cliente

```
POST /api/movies HTTP/1.1
Content-Type: application/json

{
  "title": "Matrix",
  "year": 1999
}
```

`POST /api/movies`

Identifica a operação e o endereço que o controller deve atender.

O corpo do pedido

O JSON contém os dados do filme. Content-Type indica que o corpo usa o formato application/json.

O controller precisa de receber estes valores num objeto MovieRequest.

Rumos

Jackson: conversão entre JSON e objetos Java

Jackson é a biblioteca que lê e escreve JSON. O Spring usa-a através de um conversor HTTP.

Entrada: desserialização

```
JSON do pedido

MovieRequest
```

Jackson lê os nomes e valores do JSON.
Consulta o tipo Java e as anotações
para criar e preencher o objeto.

Saída: serialização

```
MovieResponse

JSON da resposta
```

Jackson lê as propriedades do objeto
e escreve os nomes e valores no JSON,
conforme a configuração.

ObjectMapper é a API central de Jackson 2. No controller, o Spring trata da chamada.

JSON e MovieRequest descrevem os mesmos dados

O Spring lê o JSON e cria o objeto que vai entregar ao método do controller.

No corpo HTTP: JSON

```
{
  "title": "Matrix",
  "year": 1999
}
```

"title" fornece o texto do título.
"year" fornece o valor numérico do ano.

No método Java: MovieRequest

```
request.title()  // "Matrix"
request.year()   // 1999
```

request é o nome do parâmetro Java.
O método usa os valores através do DTO.

Esta conversão chama-se desserialização. @RequestBody pede ao Spring que a faça.

@RequestBody: quando o método recebe o DTO

O Spring converte o corpo HTTP antes de chamar create com o DTO preenchido.

```
@PostMapping("/api/movies")
public void create(
    @RequestBody MovieRequest request
) {
    // usar request.title() e request.year()
}
```

@RequestBody: converter

Lê o corpo e cria MovieRequest.
O HttpMessageConverter adequado usa Jackson para este JSON.

Quando a leitura falha

Se o JSON é inválido ou incompatível, o Spring devolve 400 por defeito.
O método não começa a executar.

A leitura bem-sucedida entrega valores ao método. A validação verifica se são aceitáveis.

Rumos
C2 General

Nomes diferentes: @JsonProperty e @JsonAlias

Jackson não traduz automaticamente "titulo" para title. A correspondência precisa de uma regra.

JSON enviado pelo cliente

```
{
  "titulo": "Matrix",
  "year": 1999
}
```

@JsonProperty define o nome no JSON.
O componente Java continua a chamar-se title.

Regra no DTO

```
public record MovieRequest(
  @JsonProperty("titulo")
  @JsonAlias("title") String title,
  Integer year
) {}
```

@JsonAlias aceita "title" como alternativa na entrada. Na saída, o nome é "titulo".

Sem correspondência, o campo extra pode ser ignorado ou rejeitado, conforme a configuração.

Tipos diferentes: conversão e validação

Se year é Integer, Jackson precisa de obter um número inteiro para criar o DTO.

Número inteiro

```
"year": 1999
```

Corresponde ao tipo esperado.
Ainda falta verificar se o ano
é permitido pelo contrato.

Texto com um número

```
"year": "1999"
```

Pode converter por coerção,
conforme a configuração.
O contrato deve pedir um
número.

Texto incompatível

```
"year": "ontem"
```

A conversão para Integer falha.
No Spring MVC, por defeito,
o pedido recebe 400.

@JsonProperty muda o nome. Para formatos ou conversões especiais, é preciso outra regra.

Por exemplo: @JsonFormat para datas, um desserializador próprio ou transformação na aplicação.

C2 General

Configuração da validação no Spring Boot 3+

Jackson lê os valores. Jakarta Validation verifica as regras. O Hibernate Validator executa-as.

Maven: dependência no pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Imports do DTO e do controller

```
import jakarta.validation.constraints.*;
import jakarta.validation.Valid;
```


Rumos
C2 General

Regras do MovieRequest e validação com @Valid

1. As regras ficam no DTO

```
public record MovieRequest(
  @NotBlank(message="Título obrigatório")
  @Size(max=120) String title,

  @NotNull @Min(1888) Integer year
) {}
```

2. @Valid aciona a validação

```
@PostMapping("/api/movies")
public void create(@Valid @RequestBody
                        MovieRequest request) {
  // executar a operação
}
```

Pedido inválido

```
{"title":"   ", "year":1200}
```

Por defeito: 400 Bad Request.
O Spring rejeita este pedido antes de executar o método do controller.

@NotBlank: título com texto. @Size: máximo 120. @NotNull: ano presente. @Min: ano ≥ 1888.

C2 General

Presença, texto e tamanho

Regras padrão de jakarta.validation.constraints, usadas com o Spring

Anotação	Regra	Exemplo ou cuidado
`@NotNull`	Exige uma referência não nula.	"" e " " ainda passam.
`@Null`	Exige que o valor seja null.	Útil para id antes da criação.
`@NotEmpty`	Exige texto ou contentor não vazio.	null, "" e lista vazia falham.
`@NotBlank`	Exige texto com carácter não branco.	" " falha. "Matrix" passa.
`@Size(min=2,max=80)`	Limita comprimento ou tamanho.	Texto, coleção, mapa ou array.
`@Email`	Verifica o formato de um email.	Não confirma que a caixa existe.
`@Pattern(regexp="[A-Z]{2}")`	Exige correspondência com a expressão.	"PT" passa. "Pt" falha.

@Size, @Email e @Pattern aceitam null. Combine com @NotNull ou @NotBlank se obrigatório.

Rumos
C2 General

Números: limites, sinal e casas decimais

Escolha a regra numérica e combine-a com @NotNull quando o campo for obrigatório.

Anotação	Regra e exemplo
`@Min(1888)`	Valor ≥ 1888. O limite também passa.
`@Max(2100)`	Valor ≤ 2100. O limite também passa.
`@DecimalMin("0.01")`	Valor ≥ 0.01, por exemplo num BigDecimal.
`@DecimalMax("999.99")`	Valor ≤ 999.99. inclusive=false exclui o limite.
`@Positive`	Valor > 0. Zero falha.
`@PositiveOrZero`	Valor ≥ 0. Zero passa.
`@Negative`	Valor < 0. Zero falha.
`@NegativeOrZero`	Valor ≤ 0. Zero passa.
`@Digits(integer=3,fraction=2)`	Até 3 dígitos inteiros e 2 decimais: 123.45 passa.

As regras numéricas aceitam null em tipos como Integer e BigDecimal. @NotNull rejeita-o.

Datas e valores booleanos

As regras temporais comparam o valor com o relógio do validador.

Anotação	Regra	Exemplo
`@Past`	Data ou instante no passado.	Ex.: data de nascimento anterior.
`@PastOrPresent`	Passado ou presente.	Hoje passa num LocalDate.
`@Future`	Data ou instante no futuro.	Ex.: marcação posterior a agora.
`@FutureOrPresent`	Presente ou futuro.	Hoje passa num LocalDate.
`@AssertTrue`	Exige true.	Ex.: confirmação de termos.
`@AssertFalse`	Exige false.	Ex.: indicador que deve estar inativo.

Datas: use tipos temporais, como LocalDate. Booleanos: boolean ou Boolean.

Todas estas regras aceitam null. Adicione @NotNull quando a presença for obrigatória.

@Valid e @Validated: onde usar

@Valid

No @RequestBody, pede validação do DTO.
Num campo com outro objeto, permite
validar também as regras desse objeto.

```
@NotNull @Valid
AddressRequest address;
```

@NotNull exige a presença.
@Valid verifica as regras de AddressRequest.

@Validated

Numa classe de serviço Spring, ativa
a validação dos parâmetros dos métodos.

```
@Service
@Validated
class MovieService {
  public void find(@Positive Long id) { }
}
```

```
Import: org.springframework.validation.annotation.Validated
```

Mapping: JSON e transformação entre objetos

Com o pedido válido, a aplicação transforma MovieRequest em Movie e escolhe os dados da resposta.

JSON e DTO

Jackson lê ou escreve JSON.
@JsonProperty associa nomes externos
a propriedades do objeto Java.

```
"titulo"
@JsonProperty("titulo") String title
```

DTO e modelo interno

A aplicação escolhe e transforma valores.
Nesta aula, fazemos isso manualmente
com new Movie e new MovieResponse.

```
new Movie(id, request.title(),
          request.year())
```

Aula 4: integração TMDB retoma DTO externo e mapping para manter o contrato próprio.

C2 General

Mapping explícito torna a transformação visível

```
Movie movie = new Movie(
    id,
    request.title(),
    request.year()
);

return new MovieResponse(
    movie.id(), movie.title(), movie.year()
);
```

- MovieRequest fornece os dados de entrada.
- Movie representa o estado interno.
- MovieResponse define os dados públicos.

Retorno direto: os dados tornam-se o corpo HTTP

Em @RestController, o Spring escreve o retorno no corpo da resposta.
Um HttpMessageConverter escolhe a representação. Para JSON, usamos Jackson.

Tipo Java	O que representa	Exemplo de corpo HTTP
MovieResponse	DTO: campos da resposta.	{"id":1,"title":"Matrix"}
List<MovieResponse>	Uma coleção de DTOs.	[{"id":1,"title":"Matrix"}]
Map<String, Object>	Pares de chave e valor.	{"total":1}
String	Texto simples em text/plain.	Filme criado
Integer / Boolean	Valor numérico ou lógico.	1 ou true

C2 General

ResponseEntity<T>: corpo, status e cabeçalhos

T indica o tipo do corpo. ResponseEntity acrescenta controlo sobre a resposta HTTP.

DTO direto

```
MovieResponse get() {
  return movie;
}
```

```
HTTP/1.1 200 OK
Content-Type: application/json

{"id":1,"title":"Matrix"}
```

DTO dentro de ResponseEntity

```
ResponseEntity<MovieResponse>
create() {
  return ResponseEntity
      .created(uri).body(movie);
}
```

```
HTTP/1.1 201 Created
Location: /api/movies/1
Content-Type: application/json

{"id":1,"title":"Matrix"}
```

O corpo contém o DTO em JSON. Status e cabeçalhos seguem fora do JSON.

HttpStatus: constantes e códigos HTTP

HttpStatus é um enum de org.springframework.http.
Cada constante representa um código. Exemplo: HttpStatus.CREATED vale 201.

Constante do enum	Código	Constante do enum	Código
OK	200	NOT_FOUND	404
CREATED	201	METHOD_NOT_ALLOWED	405
ACCEPTED	202	CONFLICT	409
NO_CONTENT	204	UNSUPPORTED_MEDIA_TYPE	415
BAD_REQUEST	400	TOO_MANY_REQUESTS	429
UNAUTHORIZED	401	INTERNAL_SERVER_ERROR	500
FORBIDDEN	403	SERVICE_UNAVAILABLE	503

Uso no controller

```
return ResponseEntity.status(HttpStatus.CREATED).body(created);
```


C2 General

Onde definir o status e como chega ao utilizador

1. A aplicação define o resultado

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(created);
```

Exemplo no controller: 201 e o DTO criado.
Para 204: ResponseEntity.noContent().build().

Quem trata os outros casos?

MVC trata erros de leitura/validação, como 400.
@RestControllerAdvice + @ExceptionHandler tratam exceções.
Filtros de segurança tratam 401/403 na API.
Falhas inesperadas podem resultar em 500.

2. O Spring envia a resposta HTTP

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":1,"title":"Matrix", "year":1999}
```

O Spring escreve o status e os cabeçalhos.
Jackson transforma o DTO em JSON.

3. A aplicação cliente apresenta

O cliente lê o código e o corpo.
Decide a mensagem que o utilizador vê,
por exemplo “Filme criado”.

201 Created + Location: a URI do novo recurso

1. Por que devolver o cabeçalho Location?

```
HTTP/1.1 201 Created
Location: /api/movies/42
Content-Type: application/json
```

Contrato HTTP explícito

Em 201, Location identifica diretamente o recurso criado; sem o cabeçalho, a referência recai na URI alvo da requisição.

Cliente desacoplado

O frontend lê a URL devolvida pelo servidor. Não precisa reconstruir nem “adivinhar” a rota do item recém-criado.

2. Implementação com Spring MVC

```
@PostMapping
ResponseEntity<MovieResponse> create(...) {
  var created = service.create(...);
  var location = ServletUriComponentsBuilder
      .fromCurrentRequest()
      .path("/{id}")
      .buildAndExpand(created.id())
      .toUri();
  return ResponseEntity.created(location)
      .body(created);
}
```

fromCurrentRequest() usa a rota atual; path + buildAndExpand completam a URI.

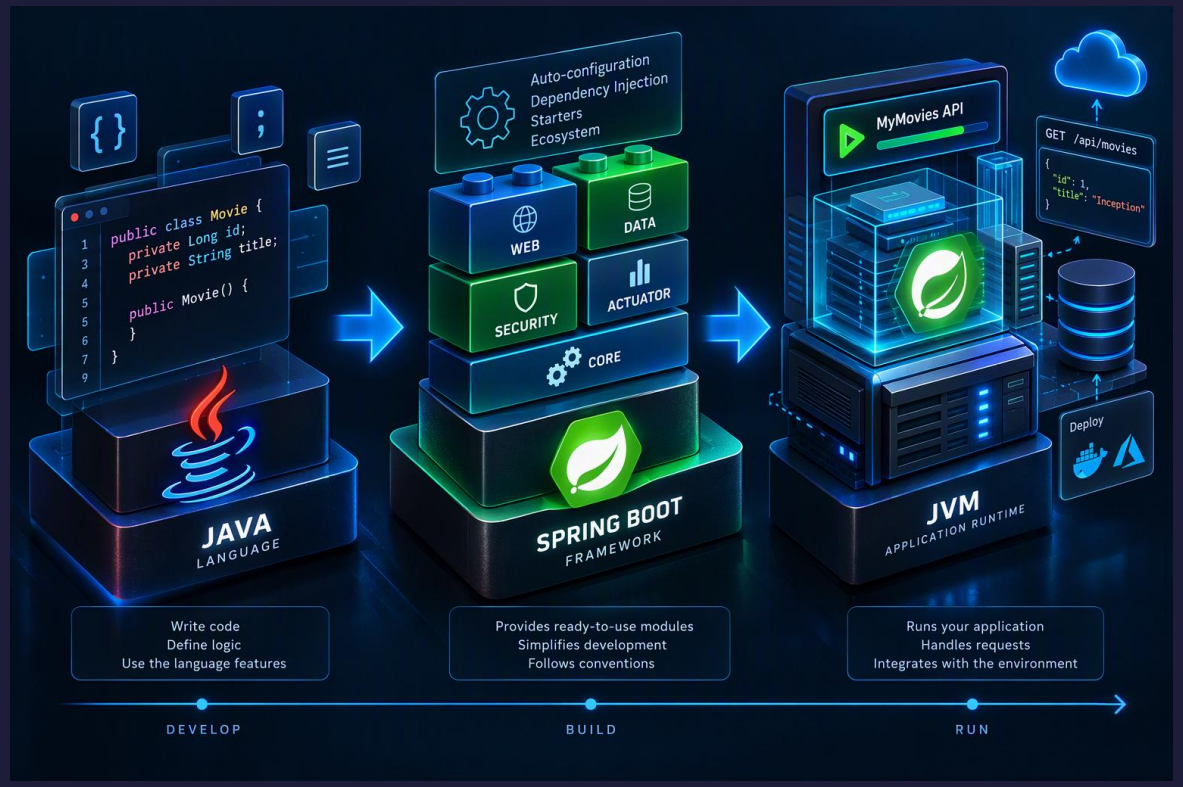
ResponseEntity.created(location)

envia 201 + Location; body(created) mantém o DTO na resposta.

Fonte: RFC 9110 §§ 10.2.2 e 15.3.2; Spring Framework — ServletUriComponentsBuilder.

Rumos

Lab: CRUD + VALIDATION



Requisito de Negócio (Estória de Usuário):

"Como utilizador da plataforma MyMovies, quero conseguir atualizar a nota de avaliação (rating) de um filme do meu catálogo sem precisar de reenviar o título ou o ano de lançamento."

Critérios de Aceite Técnicos:

1. HTTP Method & URI: PATCH /api/movies/{id}/rating

2. Corpo da Requisição: DTO RatingUpdateRequest contendo o campo `rating` (Double)

3. Regras de Validação: O rating é obrigatório (@NotNull) e deve estar entre 0.0 e 10.0 (@DecimalMin / @DecimalMax)

4. Respostas HTTP Esperadas:
   - 200 OK: Retorna o MovieResponse atualizado no corpo.
   - 400 Bad Request: Nota fora do intervalo (ex: -2.0 ou 15.0) ou nula.
   - 404 Not Found: Filme com o ID especificado não foi encontrado

C2 General

Checkpoint: aula-02-api-design

- ✓ GETs continuam funcionais
- ✓ POST válido → 201 + Location
- ✓ Payload inválido → 400
- ✓ DELETE → 204
- ✓ DTO público ≠ Movie interno

`checkpoint/aula-02-api-design`

Próxima aula: persistência com JPA/Hibernate

Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —